# WOD2Sim: Separating Integration Failures from Policy Failures in Closed-Loop Driving Simulation

Alba Maria Tellez Fernandez

***Abstract*— Closed-loop driving evaluation can misattribute degraded metrics to policy quality when a simulator boundary drops route geometry, changes timing, or produces incomplete evidence. WOD2Sim audits five contracts between Waymo Open Motion Dataset-style trajectory policies and AlpaSim external drivers. An official protocol is replayed through route-following and NAVSIM EgoStatusMLP services under full and command-only route modes. All four return 60/60 finite, nonstationary trajectories. Geometry removal triggers `semantic.command_only` for route following and changes 56/60 endpoints. It is correctly accepted for EgoStatusMLP, whose published signature excludes route geometry; all 60 paired outputs remain identical. Separately, 15 adapter sessions are paired with 15 predefined single-fault mutations while withholding labels. WOD2Sim classifies 30/30 cases, localizes 15/15 faults, and produces 0/15 control false positives; a completion gate classifies 15/30 and detects no faults. Post-parse detection is 28.096 $\mu$s median and 55.774 $\mu$s p95. Camera-set and freshness checks add 68.871 $\mu$s median and 309.613 $\mu$s p95. A reactive AlpaSim rollout closes the camera-blind learned policy through controller and physics for 197/197 finite outputs over 19.930 simulated seconds. Its camera repeats one recorded seed on a declared flat surface; this supports lifecycle and service timing only, not visual-policy quality, comparative overhead, human diagnosis time, scenario coverage, or cross-framework transfer.**

## I. Introduction

Dataset-trained trajectory policies and closed-loop simulators expose different failure surfaces. A policy designed around the Waymo Open Motion Dataset (WOMD) interface [1] may assume logged observations, local route geometry, and a fixed trajectory horizon. A simulator service must also accept incremental messages, meet the runtime clock, remain alive across session events, and retain enough evidence to explain a result. AlpaSim exposes this difference through its external-driver boundary [2].

A wrapper can therefore make an integration defect look like poor driving. Replacing a local route polyline with a left/straight/right command removes decision-relevant input, yet the rollout may still terminate with collision and progress metrics. Timing-grid mismatches, stale observations, optional import failures, missing assets, and incomplete manifests create similar confounds. A completed process is not sufficient evidence that a score characterizes policy behavior.

WOD2Sim treats this as a system-integration and attribution problem. Its contribution is threefold:

- five explicit contracts define when a simulator rollout may be interpreted as policy behavior;
- an adapter, audit, and artifact pipeline localizes violations and preserves completed, blocked, and invalid rows;
- a paired protocol-conformance suite measures localization, post-parse detector execution, and a guard-path increment against an executable status-only gate, while a recorded-message gRPC replay tests policy-specific semantic applicability across route-dependent and command-native learned policies; a separate learned-policy rollout tests live external-driver, controller, and physics feedback.

Alba Maria Tellez Fernandez is an Independent Researcher. Email: amtellezfernandez@gmail.com.

The five contract classes concern information, clocks, service state, deployment, and evidence rather than WOD records or a specific protobuf schema. A new binding maps its native policy inputs, clocks, service states, and artifacts to these contracts; the WOD-style/AlpaSim adapter is one instantiation, not the definition. This is a structural generalization hypothesis. Empirically, the present study still covers one simulator binding: its matrix uses dependency-light policies, while the replay and one reactive run add a published NAVSIM learned baseline. It does not cover a second independently maintained simulator framework.

### A. Failure Attribution Rule

A rollout permits policy-behavior attribution only if its semantic, temporal, lifecycle, deployment, and evidence contracts pass. A policy-failure attribution additionally requires a retained failure layer identifying the policy. Otherwise the row is an integration, precondition, or evidence failure. Thus a missing actor proxy is not a weak planner, a command-proxy route is not a poor route-conditioned policy, and an incomplete audit is not a collision result. This rule permits policy behavior to be examined after validation; it does not declare every degraded metric a policy failure.

## II. Contract-Gated Integration

Table I maps the five recurring boundary mismatches to enforcement mechanisms. The *semantic contract* preserves the inputs declared by the active policy, including route geometry for route-conditioned policies plus required ego, camera, and actor fields at prediction time. A high-level route command cannot substitute for a local polyline when geometry is declared. The *temporal contract* validates horizons and timestamps, anchors interpolation at the current ego pose, deterministically resamples a finite $N \times 2$ trajectory, and recomputes headings on the runtime grid.

The *lifecycle contract* makes start, active, close, cleanup, duplicate close, and late events explicit. Recoverable warnings are distinguished from fatal service errors. The *deployment*

TABLE I

CONTRACTS ENFORCED AT THE POLICY/SIMULATOR BOUNDARY.

| Mismatch | Contract | Mechanism | Validation |
|---|---|---|---|
| Command-only route | Semantic | Preserve route geometry | route-source audit |
| Policy horizon/runtime grid | Temporal | Deterministic resampling | cadence tests |
| Script flow/session service | Lifecycle | Idempotent late-event handling | lifecycle tests |
| Implicit host state | Deployment | Materialized manifests | readiness checks |
| Process exit/evidence | Evidence | Audit-valid summaries | claim gate |

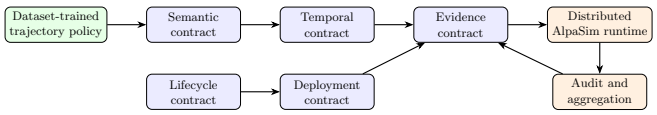


Fig. 1. Contract gates localize semantic, temporal, lifecycle, deployment, and evidence failures around the external-driver boundary.

*contract* records launch commands, scene and policy identifiers, timeout, endpoint, repository state, image identity, and gated inputs. The *evidence contract* requires unique run identifiers, immutable manifests, terminal status, route and sensor audits, failed-run retention, and hashes linking generated paper assets to aggregate rows.

Figure 1 shows the boundary. WOD2Sim reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates optional model discovery. On the runtime side it records deployment and evidence state before a row can be attributed to policy behavior. Constant-velocity and route-following models form the public core. The protocol replay downloads and hash-checks NAVSIM's official EgoStatusMLP seed-0 checkpoint without redistributing it. Other learned checkpoints and direct actor-aware entry points remain gated when their artifacts or scene-matched actor proxies are unavailable.

## III. IMPLEMENTATION AND EVIDENCE PIPELINE

### A. Policy-Facing Adapter

The external driver maintains isolated state for each simulator session. Image, egomotion, route, and close messages update that state; prediction reads one consistent snapshot rather than reconstructing inputs from process-global variables. The route converter carries simulator waypoints into the policy input and records their source. When the explicit command-only diagnostic is selected, it labels the fallback as `command_proxy`; the audit rejects the row if the active policy declares route geometry as required, even if the service and simulator finish normally. A command-native policy does not produce that false alarm. Static hazards remain geometry-bearing objects, while dynamic hazards retain position, motion, heading, and shape fields when the external interface supplies them. Visibility-risk signals remain diagnostics and do not create synthetic obstacles.

The temporal adapter maps a source trajectory to runtime times $t_k = k/f_{\mathrm{run}}$ over the requested horizon. The current ego origin is prepended before interpolation, source timestamps must be finite and strictly increasing, and the output must be a finite $N \times 2$ array. Matching grids preserve samples; mismatched grids use deterministic interpolation, after which headings are recomputed from successive runtime points.

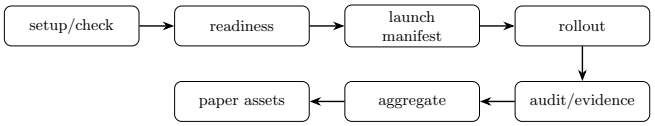


Fig. 2. Manifests and audits precede aggregation, so blocked or invalid rows remain visible and cannot silently become paper results.

The final protocol serializer independently rejects empty trajectories, non-finite coordinates or headings, and unequal point/heading counts. Invalid horizons, frequencies, grids, or trajectories therefore fail the temporal contract rather than reaching vehicle control. These behaviors are covered by dependency-light tests, but the scene-level temporal comparison remains blocked and is not reported as a result.

### B. Artifact Lineage

Figure 2 shows the evidence path. Before launch, the matrix runner expands a configuration row and materializes its command, environment, scene identity, policy identity, adapter mode, prerequisite state, and expected route source. A readiness check distinguishes an executable row from a missing checkpoint, actor proxy, asset, or runtime dependency. Terminal state is then recorded as completed, failed, blocked, or planned; non-completed rows are retained rather than removed from the denominator.

For completed simulator rows, the audit reconstructs route-source and sensor-freshness status from retained telemetry and loads behavior metrics without using them to decide route validity. Aggregation rejects duplicate completed run identifiers, preserves failure codes, forms paired rows by policy, scene, and execution identifier, and writes CSV, JSON, LaTeX tables, and vector figures. Generated paper assets carry the aggregate data hash, and the submission validator compares their values with the JSON source. This lineage prevents manual table edits from changing a paper result independently of the row evidence. Restricted sensor frames and licensed scenes are not redistributed; the public package retains manifests, aggregate metrics, audit fields, and a stable frame schema. Consequently, artifact lineage is public, but complete replay still depends on legitimately acquired external scenes and runtime software.

## IV. EVALUATION METHOD

The contract-validation matrix (CVM) contains 148 configured rows: 45 core, 30 semantic-ablation, 18 temporal-ablation, 40 lifecycle, and 15 fault-injection rows. It completes 115 rows, including 60 simulator rollouts; 33 optional-extension rows remain blocked.

The public core executes two dependency-light policies over 15 locally cached front-camera scenes. The semantic matrix pairs full-route and command-only variants of route following on the same scenes. The launcher does not expose a simulator seed override, so each policy/scene arm has one execution; the configured seed is provenance metadata rather than proof of deterministic replay. No authoritative metadata

verifies straight, intersection, lane-change, dense-traffic, occlusion, or merge labels: 15 scenes remain unclassified and 0/6 required categories are verified.

For transport-inclusive current-schema evidence, we replay the official AlpaSim integration recording identified in the artifact manifest through four live driver services. The two policy families are route following and NAVSIM's published learned EgoStatusMLP seed-0 baseline [3], [4], executed on CPU from a revision- and hash-pinned checkpoint. Its published input is planar velocity, acceleration, and a four-way command; it is neither a visual nor multimodal policy. For each policy, both arms receive the same recorded session, route, egomotion, camera, and `Drive` messages. One service mode retains all 20 route points and the other retains only the derived command. Route following declares the polyline required; EgoStatusMLP consumes the command and ego status but not the polyline. The client times the complete loopback gRPC call using `perf_counter_ns`. Each arm has one sequential execution with 60 calls. We pair calls by index, timestamps, and raw route, then measure Euclidean separation between output endpoints in the local ego frame. Because returned trajectories do not alter recorded future observations, this is a non-reactive protocol replay rather than a simulator rollout. It includes client serialization, transport, service dispatch, adapter execution, and response serialization, but not simulator stepping or feedback. Arm timing differences are not overhead estimates.

To test live feedback separately, one run uses the current public AlpaSim fixture and the same pinned EgoStatusMLP checkpoint. AlpaSim's external driver, controller, and physics services remain live; each returned trajectory affects the next ego state. Because the fixture has no collision mesh, the artifact declares an added flat $z = 0$ surface. AlpaSim calls its official `video_model` boundary, while the external endpoint repeats the fixture's recorded seed frame and logs every requested live trajectory. This is valid for the camera-blind checkpoint, whose input signature excludes pixels, but not for a visual policy. A same-scene route-following control declares camera freshness required and is expected to reject the repeated image rather than complete.

The retained external AlpaSim trace contains 197 drive calls and establishes service-interface and recorded-latency conformance. It uses an earlier telemetry schema that did not record the final finite-output check, so it is not the source of the mutation experiment. Instead, the current adapter generates 15 separate sessions containing 120 drive calls; all 120 calls explicitly record a finite serialized trajectory. The sessions vary accepted camera aliases, route geometry, speed, and valid sensor age, and each passes the five contracts before mutation.

Each clean session is paired with a copy containing one predefined single-field or runtime-context mutation. The 15 mutations cover three semantic, three temporal, three lifecycle, one plugin precondition within deployment, three other deployment, and two evidence faults. Mutation construction and detection are separate: the detector receives only events and runtime context, while the scorer retains the expected label.

TABLE II
DEPENDENCY-LIGHT PUBLIC-CORE INTEGRATION STATUS.

| Public core policy | Rows | Done/att. | Audit | Route | Sensor | Crash | Blocked |
|---|---|---|---|---|---|---|---|
| constant_velocity | 15 | 15/15 | 14/15 | 14/15 | 15/15 | 0 | 0 |
| route_following | 15 | 15/15 | 14/15 | 14/15 | 15/15 | 0 | 0 |

For a paired comparator, define the executable completion-and-metrics gate

$$B_{\text{status}}(r) = \mathbf{1}[\text{completed}(r) \wedge \text{metrics}(r)]. \tag{1}$$

Both methods return a decision for all 30 cases. We report exact classification, detection, localization, false-positive, and paired-discordance counts. We do not compute a population confidence interval or hypothesis test because the sessions and fault operators are deliberately constructed rather than sampled independently from a defined population.

Detector timing uses 200 randomized iterations and five calls per batch, yielding 3,000 fault-case and 6,000 all-case per-call batch means. The timer starts after telemetry has been parsed and excludes file I/O and human investigation. A separate 1000-iteration paired ablation rotates over 15 deterministic valid sessions and compares the in-process adapter Drive path with camera-set and sensor-freshness guards enabled or disabled; context-length validation remains in both arms. Order is randomized batched by 20. Both paths retain state-to-input assembly, prediction, trajectory serialization, finite-output validation, reasoning parsing, and in-memory telemetry; output trajectories must match exactly. Completed simulator rows are separately audited for route source and sensor freshness. The same status gate is applied to metric-bearing command-only rows, while WOD2Sim additionally requires all five contracts.

## V. RESULTS

Table II reports integration status rather than behavior scores. All 30/30 public-core rows complete with no service crash or blocker; 28 are audit-valid. The two invalid core rows and one invalid semantic full-route row come from the same scene, where the audit finds late command-proxy fallback. They remain integration/evidence failures despite successful process completion.

Designed-suite results appear in Table III. WOD2Sim is correct on 30 of 30 cases, detects and localizes all 15 faults, and flags none of the 15 controls. The status gate is correct on 15 cases because it accepts every control and every completed fault case; it detects no faults. All 15 discordant pairs favor WOD2Sim. These are descriptive counts for the declared suite, not evidence of population-level superiority to another integration framework.

Post-parse contract-gate execution is 26.169 $\mu$s median and 52.962 $\mu$s p95 over 6,000 measurements. On the 3,000 fault cases it is 28.096 $\mu$s median and 55.774 $\mu$s p95. The guarded in-process adapter Drive path is 617.549 $\mu$s median and 897.100 $\mu$s p95. Its paired guard-path increment is 68.871 $\mu$s median and 309.613 $\mu$s p95 over 1000 measurements; guarded and unchecked calls return identical trajectories and headings.

TABLE III

PAIRED PROTOCOL-CONFORMANCE DIAGNOSTICS. FP DENOTES A VALID CONTROL CLASSIFIED AS FAULTY.

| Gate | Correct | Detected | Localized | FP |
|---|---|---|---|---|
| WOD2Sim | 30/30 | 15/15 | 15/15 | 0/15 |
| Status-only | 15/30 | 0/15 | 0/15 | 0/15 |

TABLE IV

EXECUTED POLICY-FAMILY PROTOCOL REPLAY. "MOVING" MEANS ENDPOINT PROGRESS EXCEEDS 1 M; AUDIT OUTCOMES ARE RECOMPUTED FROM RETAINED TELEMETRY.

| Policy | Policy-visible route | Finite | Moving | Audit |
|---|---|---|---|---|
| Route following | 20 waypoints | 60/60 | 60/60 | pass |
| Route following | command only | 60/60 | 60/60 | semantic fault |
| NAVSIM EgoStatusMLP | command (route retained) | 60/60 | 60/60 | pass |
| NAVSIM EgoStatusMLP | command only | 60/60 | 60/60 | pass |

These timings describe the measured software paths on one host and are not end-to-end runtime or time-to-diagnosis measurements. Separately, all 197/197 calls in the retained external trace meet its 100-ms target.

Table IV reports the four current-schema replay arms. Every arm returns 60/60 finite and nonstationary trajectories and meets the 100-ms target. Both route-preserving traces produce zero diagnostics. The reduced route-following trace produces only `semantic.command_only`, whereas the reduced EgoStatusMLP trace passes because route geometry is outside its declared input signature. Thus applicability of the semantic decision is tested across 2 policy families at the same boundary.

Route removal changes the route-following endpoint by more than 0.1 m on 56/60 calls. EgoStatusMLP is the policy-signature negative control: all 60 full/reduced trajectory pairs are exactly equal, as required because the mutation removes no model input. Route-following full/reduced median/p95 client-to-service latency is 3.769/4.833 and 3.104/3.958 ms; EgoStatusMLP is 4.715/5.945 and 4.943/6.963 ms. These are descriptive values from one ordered run per arm, using different runtime images for the dependency-light and learned services; no format or policy overhead is inferred.

The learned reactive arm completes 1/1 rollout with 197/197 finite outputs and 197/197 below its 100-ms internal target. Driver-internal median/p95 is 1.982/3.362 ms; AlpaSim observes 3.206 ms mean `Drive` RPC time. The run advances 61.863 m over 19.930 simulated seconds in 16.510 active wall-clock seconds (18.900 including setup). These are measurements of this exact configuration, not overhead versus another integration. The retained behavior record includes `wrong_lane`=1.000 and is not used as a policy-quality result. In the camera-validating control, 4 calls complete before WOD2Sim rejects the next unchanged frame as stale. Thus the static camera limitation is detected rather than silently accepted for a model that requires camera freshness.

Constructing the clean sessions exposed two non-injected adapter defects. Camera-alias priority could select an older frame and falsely report staleness; the adapter now selects the freshest candidate. In the real replay, AlpaSim's dynamic-speed field became zero while consecutive poses still advanced, causing 59 stationary outputs in the superseded artifact; the adapter now falls back to pose-derived speed when those signals conflict. Regression tests cover both cases, and all four replay arms were regenerated. Integrating the checkpoint also exposed a serializer off-by-one error that replaced the first future pose with the current pose and dropped the final prediction. Telemetry schema v3 now records current and future counts separately, and regression tests require the current pose plus every predicted waypoint. None of these repaired defects is counted among the 15 injected faults.

TABLE V

CLOSED-LOOP ATTRIBUTION CHECKS FROM RETAINED ROLLOUT AND AUDIT ARTIFACTS.

| Closed-loop check | Obs. | Denom. | Meaning |
|---|---|---|---|
| Full-contract audit-valid | 42 | 45 | audit passed |
| Comparison-eligible pairs | 14 | 15 | valid/invalid arms |
| Status-only baseline accepted | 15 | 15 | completion + metrics |
| Invalid route rows rejected | 15 | 15 | route gate |
| Verified scenario categories | 0 | 6 | coverage withheld |

The closed-loop invalidation check in Table V is consistent with the mutation study. All 15/15 command-only rows satisfy $B_{\text{status}}$, whereas WOD2Sim rejects 15/15 as route-invalid. Thus metric-bearing completion alone would admit records whose route contract is known to be false.

Only 14/15 route-loss pairs satisfy the comparison criterion. Their full-minus-command means are mixed (progress -0.052, collision-any 0.071, plan deviation 0.016); they describe retained outcomes rather than a policy effect because each arm has one execution and no effective seed control.

Across all retained rows, 42 are policy-behavior-attributable diagnostics, 0 are policy-failure-attributable, and 33 are integration/precondition blockers. The remaining 106 include blocked, contract-invalid, synthetic, or benchmark-incomplete evidence. These counts operationalize the distinction: degraded metrics cannot assign policy failure unless the contracts pass and the retained failure layer is policy.

### A. Interpretation

The controlled experiment validates classification and localization for the declared mutation set. Its 15 separately instantiated clean sessions supply the paired negative cases, so the observed 0/15 false-positive count is scored from detector output rather than defined by audit admission. The sessions and mutations are nevertheless products of the same protocol design and are not independent population samples. The closed-loop rows test a second path: adding route-source validity changes the attribution of all 15 metric-bearing command-only records without changing their simulator outcome.

The timing microbenchmarks instrument two software paths. Within the in-process adapter, camera-set and freshness checks have a median increment of 68.871 $\mu$s. Post-parse detector execution has a median of 28.096 $\mu$s. A missing

actor proxy or checkpoint remains a deployment precondition, a command proxy a semantic violation, and missing audit state an evidence violation. Typed outcomes keep those cases available for debugging while excluding them from policy attribution.

## VI. Related Work and Limitations

The contracts instantiate interface reasoning for cyber-physical systems [5], [6]. Closed-loop platforms and benchmarks such as CARLA, nuPlan, Waymax, NAVSIM, and AlpaSim [7], [8], [9], [3], [2] emphasize simulation or planning evaluation. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [10], [11], [12], [13] generate or mutate conditions. WOD2Sim is complementary: it asks whether the policy/simulator boundary preserved the assumptions needed to interpret a rollout.

The abstraction boundary is broader than WOD: a route-conditioned policy in another framework can bind its intent representation to the semantic contract, its prediction and control clocks to the temporal contract, and its service and artifact states to the remaining contracts. Such a binding may reuse the audit vocabulary without sharing WOD message types. The route-following treatment and EgoStatusMLP negative control show that the semantic gate follows a policy's declared input signature rather than one policy implementation, but both use the same AlpaSim binding. Cross-simulator transfer still requires a second integration, such as a Waymax adapter, and a framework-independent fault set.

nuPlan and NAVSIM formalize planning tasks and measures; Waymax and CARLA provide controllable simulation interfaces; AlpaSim supplies the external-driver boundary used here. WOD2Sim instead applies contract assumptions and guarantees as runtime admission and artifact-lineage checks. Scenic and driving fuzzers vary scenarios to expose system behavior, whereas our mutations vary information or preconditions at the integration boundary. The approaches can be combined.

The study scope is contract conformance. The mutations and clean sessions are framework-authored, deterministic software cases rather than natural fault samples or external simulator reruns. The retained external trace establishes a separate service-conformance result, but its earlier schema omits the explicit finite-output field required by the current detector. The status-only gate is a concrete completion-and-metrics comparator, not a surrogate for CARLA, nuPlan, or another full framework. The repository is not an official WOMD interface, does not reconstruct full WOMD worlds, and has no scene-matched actor proxy or verified scenario-category coverage. One official NAVSIM learned checkpoint is hash-pinned for the replay and one reactive external-driver rollout, but is not redistributed. It is camera-blind rather than visual or multimodal and is not evaluated for policy quality. The public reactive fixture repeats a recorded seed frame and uses a declared synthetic flat physics surface. Restricted scenes remain licensed inputs; public artifacts retain telemetry, manifests, aggregates, hashes, and testable gate logic.

### A. Threats to Validity

*Internal validity.* Mutation and detector code share the declared contracts, which can favor the expected fault vocabulary. To reduce circularity, construction and detection are separate functions, expected labels are withheld from the detector, every case is scored afterward, and 15 unmutated sessions test false alarms. The exact paired counts describe only this designed set; no independence-based significance test is applied. Timing order is randomized and paired on one host, so hardware and background-load effects remain shared measurement conditions.

*Construct validity.* The guard-path ablation covers camera-set and freshness checks in the in-process adapter Drive path over 15 deterministic valid sessions. It includes input assembly, prediction, serialization, finite-output validation, reasoning parsing, and in-memory telemetry, but excludes gRPC, file I/O, simulator work, and human investigation. The detector timer begins after JSON parsing and excludes I/O and human investigation. Neither microbenchmark timer supports an end-to-end runtime or time-to-diagnosis claim. The recorded-message replay adds gRPC transport and service dispatch, but excludes simulator stepping and feedback; it therefore supports client-to-service latency, not end-to-end runtime or human diagnosis time. The separate learned rollout includes simulator stepping and feedback and therefore supports the reported duration and service timing for that configuration. Its repeated camera seed and flat ground exclude reactive visual behavior, rendering quality, and comparative overhead. All five contracts, including the plugin precondition within deployment, are mutated against generated telemetry and context rather than rerun as physical simulator faults. Contract-valid therefore means admitted to the stated evidence boundary, not driving safety or simulator correctness.

*External and conclusion validity.* The scenes were selected by local asset availability and remain category-unclassified. The diagnostic cases use the same adapter, generator, and contract vocabulary, and the closed-loop route pairs have one execution per arm with no effective seed control. Accordingly, the descriptive comparison concerns only the declared mutations and completion gate; the measured software timings concern this host and the declared paths. The learned replay measures input-signature invariance under route removal; the reactive learned run covers one scene with synthetic ground and non-reactive pixels. Neither establishes safety, policy quality, or cross-dataset generalization. Rare-event coverage, human debugging time, and comparison with independently maintained frameworks require separate experiments. The contract taxonomy may transfer structurally, but cross-simulator empirical generalization remains untested.

## VII. Conclusion

WOD2Sim formulates a closed-loop policy boundary as semantic, temporal, lifecycle, deployment, and evidence contracts. On 15 designed faults paired with 15 valid current-adapter sessions it classifies 30/30 cases, localizes 15/15 faults, and differs descriptively from the completion-and-metrics gate on all 15 fault cases. Post-parse detector execution, a paired

guard-path increment, and current-schema client-to-service latency are measured. The semantic decision is also replicated across route following and a published learned command-native policy: only the route-dependent policy is rejected when geometry is removed, and the learned negative control remains invariant. A separate learned rollout completes the live AlpaSim driver/controller/physics loop under its declared camera-blind input contract, while a camera-validating control rejects the repeated frame. Comparative runtime overhead, human diagnosis, learned-policy quality, visual behavior, cross-framework generalization, and framework superiority are not claimed. Together with the retained closed-loop route invalidations, the result shows that explicit gates make integration evidence auditable before a rollout is attributed to policy behavior.

## ACKNOWLEDGMENT

OpenAI Codex (GPT-5) assisted with language editing, code audit, and test implementation. Repository scripts generated the measurements; the author reviewed the method, results, and manuscript and accepts responsibility for the content.

## REFERENCES

[1] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. R. Qi, Y. Zhou, Z. Yang, A. Chouard, P. Sun, J. Ngiam, V. Vasudevan, A. McCauley, J. Shlens, and D. Anguelov, "Large scale interactive motion forecasting for autonomous driving: The Waymo open motion dataset," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9710–9719, Oct. 2021.

[2] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. Garcia Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. Martinez Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, "AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving." Software, Oct. 2025.

[3] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. Datasets and Benchmarks Track.

[4] D. Dauner and Autonomous Vision Group, "NAVSIM baseline checkpoints." Hugging Face model repository, Sept. 2024. Commit 32d89c0; Apache-2.0.

[5] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, "Taming Dr. Frankenstein: Contract-based design for cyber-physical systems," *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.

[6] L. de Alfaro and T. A. Henzinger, "Interface automata," in *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 109–120, ACM, 2001.

[7] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.

[8] N. Karnchanachari, D. Geromichalos, K. S. Tan, N. Li, C. Eriksen, S. Yaghoubi, N. Mehdipour, G. Bernasconi, W. K. Fong, Y. Guo, and H. Caesar, "Towards learning-based planning: The nuPlan benchmark for real-world autonomous driving," in *2024 IEEE International Conference on Robotics and Automation*, pp. 629–636, IEEE, May 2024.

[9] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. Datasets and Benchmarks Track.

[10] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, "Scenic: A language for scenario specification and scene generation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.

[11] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, "VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems," in *Computer Aided Verification*, vol. 11561 of *Lecture Notes in Computer Science*, pp. 432–442, Springer, 2019.

[12] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1753–1767, ACM, Nov. 2022.

[13] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, "Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks," in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.